

QP4.1 – Linguagens de Programação: Questões pré-classe para a aula 1 de "Tipos de Dados"

Total de pontos 12/12

E-mail *

ramonsoares@discente.ufcat.edu.br

0 de 0 pontos

Nome *

Para registro das respostas

RAMON SOARES MENDES DE MENESES LEITE

3 de 3 pontos

São exemplos de tipos primitivos da linguagem C, exceto: *

1/1

- char
- boolean
- double
- ponteiro
- int

Assinale a alternativa correta. *

1/1

- Um tipo, para ser considerado primitivo, deve poder ser implementado diretamente no hardware, que deve oferecer operações para manipulá-lo.
- Em C, char é um tipo primitivo, enquanto que unsigned char não é um tipo primitivo.
- Dois tipos primitivos de uma mesma linguagem e com o mesmo tamanho (número de bytes) são equivalentes.
- Duas linguagens que oferecem um mesmo tipo primitivo (por exemplo, inteiro) podem oferecer faixas de valores diferentes para aquele tipo.
- Em C, um vetor é um tipo primitivo.
- Todas as linguagens possuem os mesmos tipos primitivos, variando entre os tipos não-primitivos (estruturados) oferecidos.

Em um tipo String com tamanho estático...

1/1

- Não é possível modificar o tamanho de uma string (aumentando ou diminuindo), uma vez criada.
- Todas as demais alternativas estão corretas.
- Não é possível declarar strings com tamanhos diferentes.
- Não é possível modificar o conteúdo de uma string, uma vez criada.

3 de 3 pontos

Em C, Strings são implementadas como vetores de caracteres. Considere as afirmativas abaixo:

- O espaço em memória ocupado por uma string é constante, ainda que o conteúdo da string (caracteres efetivamente usados) seja menor.
- Em strings terminadas por nulo (character '\0'), uma string inicia na posição 0 do vetor e termina na posição imediatamente anterior àquela que possui o caracter '\0'.
- Uma string terminada por nulo implementa uma string de tamanho dinâmico.
- Em uma string terminada por nulo, a função strlen (tamanho da string) recupera o tamanho da string na primeira posição do vetor de caracteres.
- A função strcpy copia o conteúdo de uma string terminada com nulo para outra string. A assinatura dessa função é: char *strcpy(char *dest, const char *src). Após a execução dessa função, o espaço ocupado em memória pela string dest será o mesmo da string src.

Quais das afirmativas estão corretas? *

1/1

- 3
- nenhuma
- 1, 2 e 3
- 4 e 5
- todas
- 1 e 2
- 3 e 5
- 2 e 4
- 2 e 5

Em C, se passarmos como parâmetro para uma função (por exemplo, strcpy) que espera uma string terminada em nulo, um vetor de caracteres normal então *

1/1

- A função processará o vetor até encontrar o último caracter.
- A função funcionará corretamente. Entretanto, o contrário - passar uma string terminada em nula para uma função que espera em vetor de caracteres - é capaz de causar erros de execução.
- A função provavelmente irá ler posições da memória além daquelas ocupadas pela string, causando um erro imprevisível no programa.

Um tipo imutável é um tipo no qual *

1/1

- É um tipo cujo tamanho ocupado em memória é fixo.
- A variável associada ao tipo não pode passar a referenciar (valor_) uma outra posição de memória.
- O conteúdo armazenado em um dado do tipo não pode ser modificado durante todo o tempo de vida da variável.

1 de 1 pontos

Sabendo que em Java o tipo String é imutável e tanto s1 como s2 são Strings, considere o código abaixo:

s2 = "abc";
s1 = "123";
s2 = s1;
s1 = "345";

Considere as seguintes afirmativas a respeito do código:

- O valor de s2 ao final do código é "123".
- A atribuição s2 = s1 modifica o conteúdo da string s2 (originalmente "abc").
- Ao final do código, há três áreas na memória com as strings "abc", "123" e "345".

Quais afirmativas estão corretas? *

1/1

- 1 e 3.
- apenas 3.
- 2 e 3.
- apenas 2.
- 1 e 2.
- apenas 1.

3 de 3 pontos

Na implementação de um jogo de cartas, um programador decidiu definir os naipes da seguintes maneira:

const int PAUS = 0;
const int COPAS = 1;
const int OURO = 2;
const int ESPADAS = 3;

Essas definições são constantes. Assim, dentro do código, o programador define uma carta da seguinte maneira:

struct Carta {
int naipe;
int numero;
} carta;

carta.naipe = PAUS;
carta.numero = 9;

Quais são os problemas dessa implementação? *

1/1

- A declaração da struct Carta não expõe claramente os valores aceitos no campo naipe.
- Quaisquer operadores de inteiros podem ser aplicados ao campo naipe, podendo criar causar problemas de confiabilidade no código.
- Todos os problemas mencionados se aplicam ao código.
- Ao campo naipe pode ser atribuído qualquer inteiro, podendo causar problemas de confiabilidade no código.

Para corrigir o(s) problema(s) da implementação de naipes da questão, o programador deveria: *

1/1

- Declarar a struct Carta como um tipo enumerado.
- Declarar um tipo enumerado cujos valores possíveis são PAUS, COPAS, OURO e ESPADAS.
- Declarar PAUS, COPAS, OURO e ESPADAS como um tipo união discriminada.
- Declarar a struct Carta como uma união discriminada.
- Declarar um tipo união discriminada cujos valores possíveis são PAUS, COPAS, OURO e ESPADAS.
- Declarar a struct Carta como uma classe.
- Declarar PAUS, COPAS, OURO e ESPADAS como um tipo enumerado.

Qual é a diferença entre um vetor bidimensional de caracteres e um vetor de strings? *

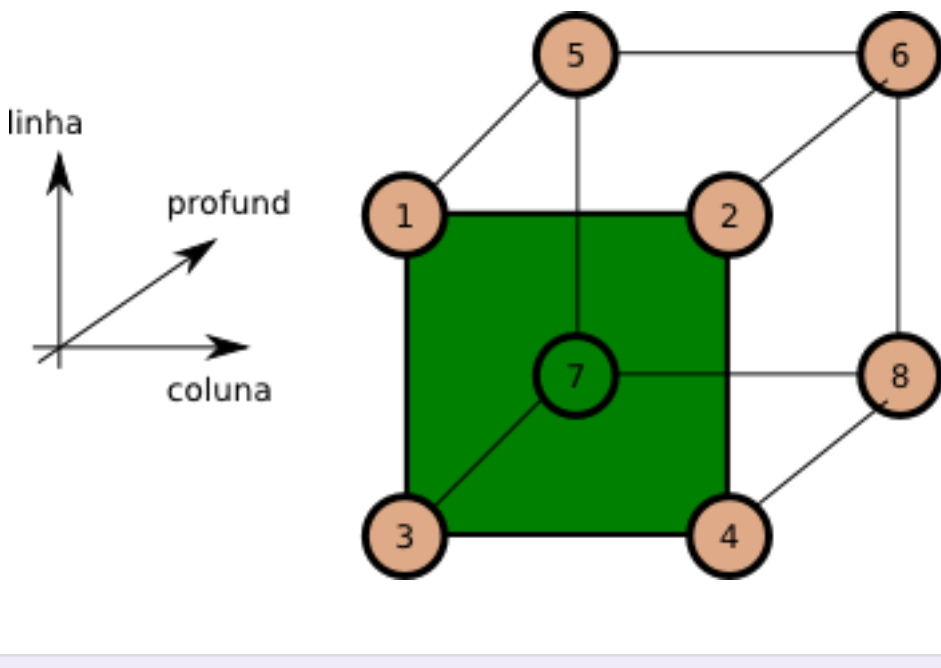
1/1

- Facilidade do acesso aos elementos.
- Vetor bidimensional não armazena strings terminadas em null, apenas caracteres.
- Vetor de strings armazena e aloca strings de tamanho variável, enquanto o vetor bidimensional aloca um tamanho fixo.
- Nenhuma

1 de 1 pontos

Desenhe a distribuição de memória dos elementos da matriz 2x2x2, considerando a implementação de matrizes em C

(o eixo "linha" está invertido)



A distribuição dos elementos em memória corresponde a: *

1/1

- 1 5 2 6 3 7 4 8
- 1 2 5 6 3 4 7 8
- 1 2 3 4 5 6 7 8
- 1 3 2 4 5 7 6 8

1 de 1 pontos

Qual das operações abaixo calcula corretamente a posição do elemento '(x,y,z)' em memória?

```
/* Operação 1 */  
addr(a[x][y][z]) = addr(a[0][0][0]) +  
x*sizeof(item) +  
y*(sizeof(item)*dimY) +  
z*(sizeof(item)*dimZ)  
  
/* Operação 2 */  
addr(a[x][y][z]) = addr(a[0][0][0]) +  
x*sizeof(item) +  
y*dimZ*sizeof(item) +  
z*dimX*sizeof(item)  
  
/* Operação 3 */  
addr(a[x][y][z]) = addr(a[0][0][0]) +  
x*dimY*dimZ*sizeof(item) +  
y*(sizeof(item)*dimZ) +  
z*(sizeof(item))  
  
/* Operação 4 */  
addr(a[x][y][z]) = addr(a[0][0][0]) +  
x*dimY*sizeof(item) +  
y*dimZ*sizeof(item) +  
z*(sizeof(item))  
  
/* Operação 5 */  
addr(a[x][y][z]) = addr(a[0][0][0]) +  
x*dimY*sizeof(item) +  
y*sizeof(item) +  
z*dimX*sizeof(item)
```

A operação que calcula corretamente a posição do elemento '(x,y,z)' em memória é *

1/1

- operação 1
- operação 2
- operação 3
- operação 4
- operação 5
- nenhuma das operações